A PRELIMINARY MINI PROJECT REPORT ON

"Placement Preparation Platform"

SUBMITTED TOWARDS THE PARTIAL FULFILLMENT OF THE REQUIREMENTS OF

BACHELOR OF TECHNOLOGY (Third Year B. Tech.)

Academic Year: 2025-26

By:

Name	PRN
Ranjit Chavan	123B1B103
Prathmesh Doiphode	123B1B110

Under The Guidance of

Prof. Shailesh B. Galande

DEPARTMENT OF COMPUTER ENGINEERING,

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

SECTOR 26, NIGDI, PRADHIKARAN

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING,

CERTIFICATE

This is to certify that, the mini project entitled

"Placement Preparation Platform"

is successfully carried out as a mini project for Full Stack Development Laboratory and successfully submitted by following students of "PCET's Pimpri Chinchwad College of Engineering, Nigdi, Pune-44".

Under the guidance of Prof. Shailesh B. Galande

In the partial fulfillment of the requirements for the Third Year B. Tech. (Computer Engineering)

Name	PRN
Ranjit Chavan	123B1B103
Prathmesh Doiphode	123B1B110

Prof. Shailesh B. Galande

Project Guide

ABSTRACT

One big problem today: job hunting feels like wandering blind. Here comes a tool built just for engineering students aiming at placement season. Not scattered PDFs or random videos - this pulls everything into one live space. Picture logging in each morning to fresh tasks that match your target role. Instead of guessing what matters, it lays out clear steps through core topics. Some days you tackle sorting methods, others dive into SQL queries or deadlock scenarios. It mixes short readings with hands-on puzzles so boredom rarely sticks around. No more jumping between five tabs trying to piece together a plan. Each section grows harder only when you're ready, never before. What once felt fragmented now moves like a steady climb. Students land here after hearing peers actually got offers using it. Behind the scenes, every feature bends toward real interview patterns. Practice does not mean memorizing - it means doing under realistic conditions. Even networking ideas show up woven into system design drills. You build rhythm because progress tracks itself without extra effort. Hard skills sharpen quietly while confidence builds from small wins. This kind of prep was missing until someone stitched it all together.

Built on React.js up front, it runs Vite behind the scenes to speed things up. Styling flows with TailwindCSS, making screens adapt smoothly across devices. Moving between pages feels natural thanks to React Router handling links. Over on the server side, Node.js powers the logic through Express.js. Data lives in MongoDB, keeping everything flexible and scalable. Users log in securely, their activity tracked by subject as they advance. Practice happens under time pressure with mock tests designed to mimic real conditions. A personalized dashboard highlights gaps in knowledge, guiding focus where needed most.

This project aims to build a platform students can easily use, one that grows with demand while connecting classroom knowledge to real-world job skills. Built to adapt over time, it could later include features like recorded lessons, training paths tailored to specific employers, or smart tools that track progress using artificial intelligence.

Index Page –

List of Figures -

List of Tables -

CHAPTER 1: INTRODUCTION

a. Problem Statement

Picking up study guides for placement exams means jumping between sites, videos, and old notes. One moment you're on a blog, next you're scrolling through video playlists – none of it fits together. A clear path built around topics like math problems, programming tasks, and subject basics just does not exist anywhere yet. Time meant for learning gets lost hunting down PDFs or sample questions instead. Each search eats minutes that add up fast. Without regular check-ins showing what's clicking and what isn't, gaps stay hidden until it's too late. Missing those blind spots slows real growth more than most admit.

b. Project Idea

Picture a website where job prep feels less like work. Instead of hunting across sites, everything sits together – math puzzles, coding tasks, computer basics – all sorted neatly. One spot holds it all. Think guided practice without clutter. Learning paths shape up around what you need. No extra noise, just material that matters. Each section links to the next, smooth as walking down stairs. Practice shows progress. Mistakes lead somewhere useful. Tools adapt quietly, never shouting features. Clarity comes first, always. Growth happens when effort meets direction

Study guides come split by subject notes cover core ideas in Data Structures & Algorithms, one piece at a time. Concepts unfold clearly in Operating Systems, built step by step. Each section on DBMS moves forward without skipping basics. Computer Networks gets explained through steady examples. OOPs appears in chunks that make sense alone, yet link together later.

Get ready – questions pop up sorted through topics, split by how tough they are. Each batch shifts on its own, sliding into place without warning. One moment it is science, next thing you face math, always changing lanes. Difficulty steps climb or drop like random stairs with no pattern. Subjects twist apart, then regroup mid-flow. Every click reshuffles the deck under new rules.

Timing your practice runs helps mimic actual test conditions. One way is setting strict limits, just like the real thing. This kind of setup builds familiarity through repetition under pressure. Working against a clock sharpens pacing skills naturally over time. Getting used to deadlines reduces surprises on exam day.

One person at a time shows how far they've come, pointing out where things get tricky. Each learner moves through tasks while weak spots quietly rise to the surface. Improvement comes into view when effort meets clear feedback. Where confusion lingers, growth begins.

Getting started means signing up, then logging in later to keep track of practice sessions. A student's progress sticks around automatically after each visit. Personal records

update without needing extra steps every time. Signing in again brings back where they left off before.

c. Motivation

Out there among classmates, placement time brings real stress. Watching others scramble through study materials, lost across websites without clear direction – that sparked something. A sense of disorder stood out, especially when results meant so much. One thing led to another after seeing how scattered everything felt. Some used five different sites at once just to practice basics. Feedback? Almost never timely, rarely useful. That gap became obvious early on. Then came the idea: what if it all lived in one place. Digital learning keeps growing anyway. Most people now have phones, laptops, some kind of link online. Learning tools should follow where life already moves. A simple site could reach far beyond city limits or campus walls. Coaching isn't cheap. Subscriptions add up fast. But open access changes who gets a shot. Distance shrinks when pages load quickly anywhere. Anyone ready to prepare deserves equal footing.

This project gives the team hands-on experience building web applications using React.js, Node.js, Express.js, and MongoDB – putting their skills into action through work that matters. While coding, they connect frontend and backend systems in ways that serve actual needs. Instead of just practicing concepts, they create something functional. Through this effort, learning happens naturally alongside tangible results. The tools come together not by chance but through focused problem solving. Real users benefit while developers sharpen their abilities.

d. Scope

The placement preparation platform covers these areas Signing up, getting in, staying out handling accounts happens through profiles guarded by token keys. Each step ties together without extra links crowding the flow. Access stays locked down unless credentials match what the system expects. Moving between states feels smooth, never clunky. Start strong with clean notes on key computer science topics, broken down by chapter. One idea at a time, built around clear explanations that fit how you learn. Each section stands apart, shaped for understanding without clutter. Focus stays sharp through structured layout and steady flow. Learning moves forward step by step, never rushed. Ideas link naturally, guided by logic instead of noise.

Take a quiz anytime. Questions split into topics show up fast. Timer runs while you pick answers. Get scores right after finishing. Each result includes clear reasons behind correct choices. Take practice exams that mimic real placement tests. These run start to finish like the actual thing. One part checks your answers right away. After finishing, you get a breakdown of how you did. Each test wraps up with clear feedback on performance. Picture your results on a screen – scores show up as bars, each subject gets its own graph. Topic progress appears in slices, like pieces of a pie slowly filling. One glance tells how much you've done so far. Charts shift as you go, updating without

asking. Numbers turn into shapes that move over time. What feels slow today looks different tomorrow when patterns start showing.

Built to fit any screen size, this system works just as well on a phone as it does on a big monitor. Whether someone uses a laptop during work hours or scrolls through pages on their handheld device late at night, everything stays clear and functional. Each view adjusts automatically, making navigation smooth no matter the gadget involved.

One step ahead could mean adding a built-in compiler so users can write code on the spot. Imagine watching a lesson right inside the app through embedded videos. Some paths might focus only on what one company looks for during hiring. Smarter suggestions may come from AI that learns how each person studies best.

e. Literature Survey / Requirement Analysis

Looking at a few current systems came first when setting up the plan

1. Start strong with GeeksforGeeks – loads of lessons and coding drills sit at your fingertips. Yet here's the catch: no clear mock exams shaped around college placement needs. Progress maps? Missing too. Tailored feedback for university prep won't show up there. Structure slips through the cracks, even if content packs a punch.
2. Practice coding problems well on LeetCode – yet skip it for aptitude, verbal skills, or deep computer science concepts.
3. Outdated design slows things down at IndiaBix – still useful for math drills, yet lacks a custom tracking system. Progress slips through the cracks without tailored feedback built in. Each session feels static, like flipping pages of an old notebook missing live updates.
4. What PrepInsta does best is train users for particular companies' interviews – yet access to deeper material means paying. Though helpful, full features aren't free.

Looking at these findings, what stands out is needing zero-cost study resources. A sleek up-to-date interface shows up as essential. Progress monitoring built right into the system matters too. Practice exams with answers and time limits come through clearly. The setup should support new features later without hassle.

CHAPTER 2: PROJECT DESIGN

b. Hardware, Software, Resources & Requirements

Hardware Requirements –

Component	Minimum Specification
Processor	Intel Core i3 / AMD Ryzen 3 or higher
RAM	4 GB (8 GB recommended)
Storage	10 GB free disk space
Network	Broadband Internet Connection (1 Mbps or higher)
Display	1280 x 720 resolution or higher

Table 1 - Hardware Requirements

Software Requirements –

Software / Tool	**Version / Description**
Operating System	Windows 10/11, macOS, or Linux (Ubuntu 20.04+)
Node.js	v18.x or higher (Runtime environment for backend)
npm	v9.x (Package manager)
React.js	v19.x (Frontend UI library)
Vite	v8.x (Frontend build tool and dev server)
TailwindCSS	v3.x (Utility-first CSS framework)
React Router DOM	v7.x (Client-side routing)
Axios	v1.x (HTTP client for API calls)
Zod	v4.x (Schema validation)
Express.js	v4.x (Node.js web framework for backend)
MongoDB	v6.x (NoSQL database)
Mongoose	v8.x (MongoDB ODM)
VS Code	v1.8x (Code Editor)
Postman	Latest (API testing tool)
Git & GitHub	Version control and repository hosting

Table 2 - Software Requirements

b. Dataset Design

Inside the system, no outside data feeds are tapped into. Built just for this setup, MongoDB holds everything in custom groupings. Main ones include:

Inside the Users Collection, name sits beside email plus a scrambled version of the password. Profile details tag along with bits tracking how far someone has gone. Each piece links without standing out too much.

One thing comes first: questions live inside documents. Every piece holds the actual question plus possible answers 11 abelled A through D. The right choice sits there too, clear and fixed. An explanation follows, showing why that answer works. Tagged by topic like DSA, DBMS, or OS it groups related ideas together. Difficulty marks whether it's Easy, Medium, or Hard. Last detail: a label tells if it's an MCQ or Aptitude type.

Test setups live here – things like time limits, groups of questions, minimum scores needed to pass, also what each exam is called. Stored together so they can be reused later when required.

Every try gets saved who took it, which test, what answers they gave, how many points they earned, how long it lasted, when it happened. A log keeps track of every session automatically.

Topic details sit inside a list each one tagged with its name, material in plain or formatted text, what subject it belongs to, plus how far into the book you are. That spot holds everything needed for review.

c. Hours Estimation

Module / Task	**Estimated Hours**
Requirement Analysis & Planning	1 hours
UI/UX Design & Wireframing	2 hours
Frontend Development (React + TailwindCSS)	1 hours
Backend Development (Express.js REST APIs)	2 hours
Database Design & Integration (MongoDB)	0.5 hours
Authentication Module (JWT)	0.5 hours
Quiz & Mock Test Logic	1 hours
Dashboard & Progress Tracking	0.5 hours
Testing & Debugging	0.5 hours
Documentation & Report Writing	1 hours
Total	**10 hours**

Table 3 - Hours Estimation

CHAPTER 3: MODULE DESCRIPTION

Block Diagram & Module Explanation

Shown in Figure 1, the block diagram outlines how the three layers connect. A typical three-part client-server setup guides the system's'design

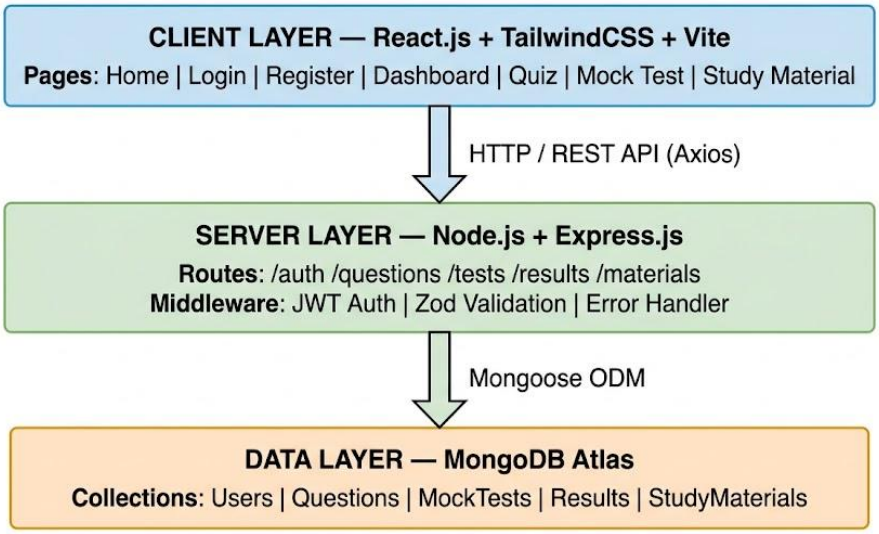


Figure 1: System Architecture Block Diagram

Module Descriptions

1.IENT LAYER — React.js + TailwindCSS + Vite

c. This sits right up front where people see and touch it every day. Built with React.js version 19 shaping how things behave, TailwindCSS version 3 handling layout flow across devices, Vite version 8 running the local setup plus packaging everything down. Pages living here include these:

Start here - his page shows what the platform does, plus ways to move around it.

Start here if you need access. Fill out the login or sign-up fields. Each entry checks itself using Zod rules right in your browser. Mistakes highlight before sending. Smooth, silent verification happens every time someone types. Protection built into each box. No extra steps needed. What you see first: a snapshot of how far someone has come. Progress bars show movement over time. Test results appear in order, one after another. Charts break down scores by topic. Each section updates automatically. Performance trends become visible week by week. Past attempts are listed clearly. No clutter, just what matters.

Take a quiz per subject. Each one gives quick results while timing your answers separately. A full practice run, built just like the real exam, keeps time ticking. Moving between questions happens through a sidebar tracker that stays visible. Each section flows without breaks, matching actual conditions closely.

2. Http Rest Api Axios Communication Bridge

From the Client Layer to the Server Layer, an arrow shows how they talk through a RESTful API. When the client reaches out, it uses standard HTTP methods like GET, POST, PUT, or DELETE. Sitting inside, Axios runs with a preset base address so every call knows where to go. It also carries built-in interceptors - hese quietly add login tokens before any message leaves. Errors coming back get caught early, handled once, across the whole system.

3. Server Layer with Node.js and Express.js

Here lives the core of operations. When a request arrives from the user interface, work begins behind the scenes. Processing happens step by step, following clear guidelines shaped by needs. Rules guide every decision made in this space. Outputs form only after careful evaluation. What comes out depends on what came in, filtered through logic. This section holds components that manage these tasks

4.Mongoose ODM Connects App to Database

d. From Server Layer to Data Layer, an arrow stands for Mongoose ODM. This tool shapes app data through schemas. Instead of writing direct queries, the server works with JavaScript objects. With it, type checks happen before reaching the database. Rules make sure only valid entries get stored. Behind the scenes, structure stays consistent. Every piece follows predefined forms. Validation kicks in automatically each time.

5. Database Layer Using MongoDB

e. Down at the lowest level, everything your app saves lives here. Hosting the data in the cloud, MongoDB Atlas steps in as the chosen NoSQL option. Key groups of information take shape through specific collections built for structure Person details live here - –ames, emails, passwords made safe through hashing. Profile bits tag along too, tucked beside each account. What sticks is a clean setup: one entry per person, built to last without fuss.

What's' inside? Multiple choice questions come with choices, right answer tucked in, a clear why it makes sense, topic label pinned on, varying challenge grades stacked under. Packed tight. Inside MockTests lives the setup for each quiz: how long it lasts, which questions show up, also what score counts as passing.

Each try gets saved under the person's'name - howing how many points they got, how long it took, what answers they gave, along with when it happened. Topic details live inside StudyMaterials, sorted by subject type along with their chapter count.

f. Comparative Study –

Feature	**Our Platform**	**GeeksforGeeks**	**LeetCode**	**IndiaBix**
Free Access	Yes (Full)	Partial	Partial	Yes
Aptitude Practice	Yes	Yes	No	Yes
Core CS Theory	Yes	Yes	No	No
Timed Mock Tests	Yes	Limited	No	No
Progress Dashboard	Yes	Limited	Yes	No
Coding Problems	Planned	Yes	Yes	No
Modern UI	Yes	Moderate	Yes	No
Company-Specific Prep	Planned	Yes	Yes	Limited

Table 4 - Comparative Study

CHAPTER 4: RESULTS & DISCUSSION

a. source Code (Key Snippets)

Snippet 1 — Frontend – App Router Setup (React Router DOM v7)

```
import { BrowserRouter, Routes, Route } from 'r‘act-router-dom';’import Home from
'.‘pages/Home';’import Login from '.‘pages/Login';’import Register from
'.‘pages/Register';’import Dashboard from '.‘pages/Dashboard';’import Quiz from
'.‘pages/Quiz';’import MockTest from '.‘pages/MockTest';’import ProtectedRoute
from '.‘components/ProtectedRoute';’
function App() {
 return (
  <BrowserRouter>
   <Routes>
    <Route path="/” ”lement={<Home />} />
    <Route path="/”ogin" ”lement={<Login />} />
    <Route path="/”egister" ”lement={<Register />} />

    <Route
     path="/”ashboard"
  element={
      <ProtectedRoute>
       <Dashboard />
      </ProtectedRoute>
     }
    />

    <Route
     path="/”uiz/:subject"
”     element={
      <ProtectedRoute>
       <Quiz />
      </ProtectedRoute>
     }
    />

    <Route
     path="/”ocktest/:id"
”     element={
      <ProtectedRoute>
       <MockTest />
      </ProtectedRoute>
     }
    />
   </Routes>
```

```
    </BrowserRouter>
  );
}

export default App;
```

Snippet 2 — Backend – Express Route for Fetching Questions

```
const express = require('e‘press')’
const router = express.Router();

const Question = require('.‘/models/Question')’
const auth = require('.‘/middleware/auth')’

// GET /api/questions?subject=DSA&difficulty=Medium&limit=10
router.get('/‘,’auth, async (req, res) => {
  try {
    let { subject, difficulty, limit = 10 } = req.query;

    // Convert limit safely
    limit = Math.min(Number(limit) || 10, 50); // max 50 questions

    // Build filter dynamically
    const filter = {};
    if (subject) filter.subject = subject;
    if (difficulty) filter.difficulty = difficulty;

    const questions = await Question.find(filter)
      .limit(limit)
      .select('-‘orrectAnswer')’ // hide correct answers

    res.status(200).json({
      success: true,
      count: questions.length,
      questions,
    });

  } catch (err) {
    console.error(err);
    res.status(500).json({
      success: false,
      message: 'S‘rver Error',’    });
  }
});

module.exports = router;
```

b. Screenshots (GUI Description)

1. Home Page

The home page features a clean and simple layout with the platform name “Placement Preparation Platform (PPP)” and a navigation bar at the top. A main heading “Master Your Next Interview” highlights the purpose, followed by a brief description of AI-based interview preparation. A search bar allows users to enter a company name and analyze interview insights, with an “Analyze” button for quick action. Login and Register options are available at the top-right for user access. The design is minimal and easy to navigate.

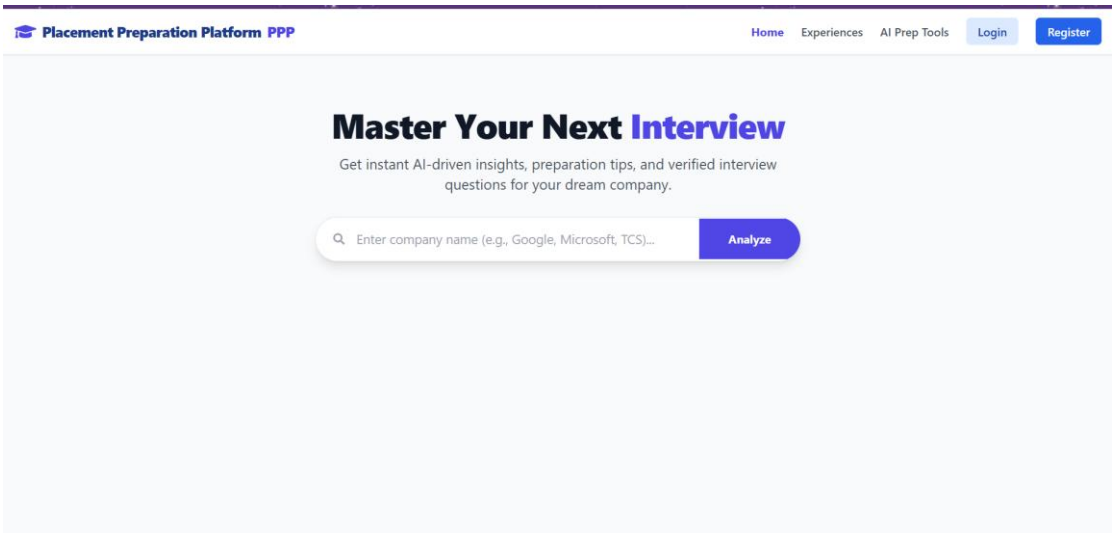


Figure 2. Home Page

2. Company Analysis Page

This page displays the analysis of a selected company after searching. At the top, a search bar shows the entered company name with an “Analyze” button. Below, a company profile card (e.g., TCS) presents an overview of the organization. The page includes key sections such as Overview, Interview Process, and Preparation Tips, providing insights into the company, hiring stages, and useful preparation guidance. A navigation bar and user options like Logout are available at the top, making the page informative and easy to use.

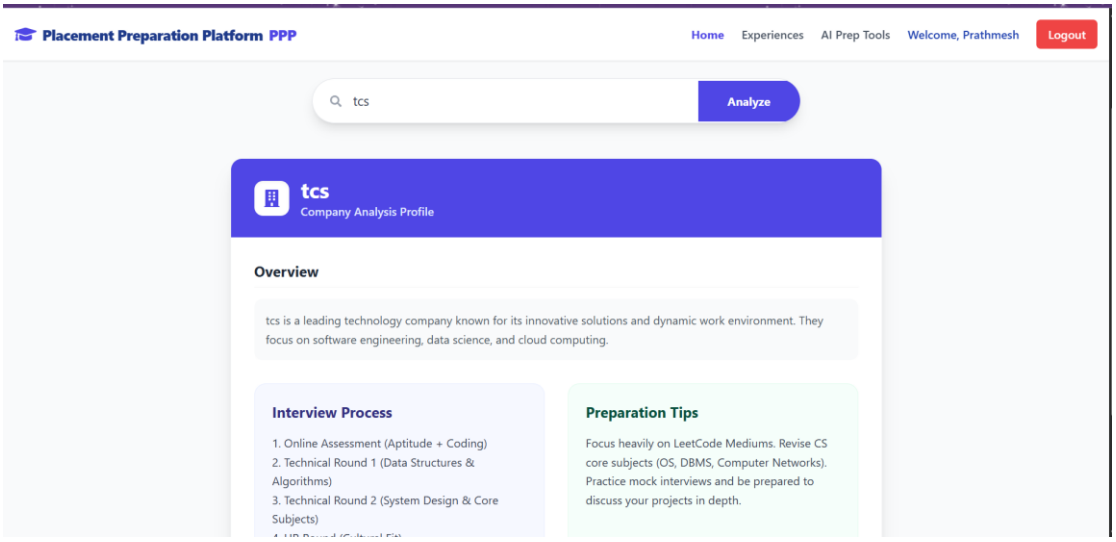


Figure 3. Company Analysis Page

3. Interview Preparation Dashboard

This page provides AI-generated mock interview questions for the selected company (e.g., *TCS*). A heading "Interview Preparation Dashboard" is displayed at the top along with the selected difficulty level. The dashboard is divided into two sections: Technical Questions and HR/Behavioral Questions. The technical section includes coding and system design challenges, while the HR section contains commonly asked behavioral questions. The layout is organized and helps users practice effectively for interviews.

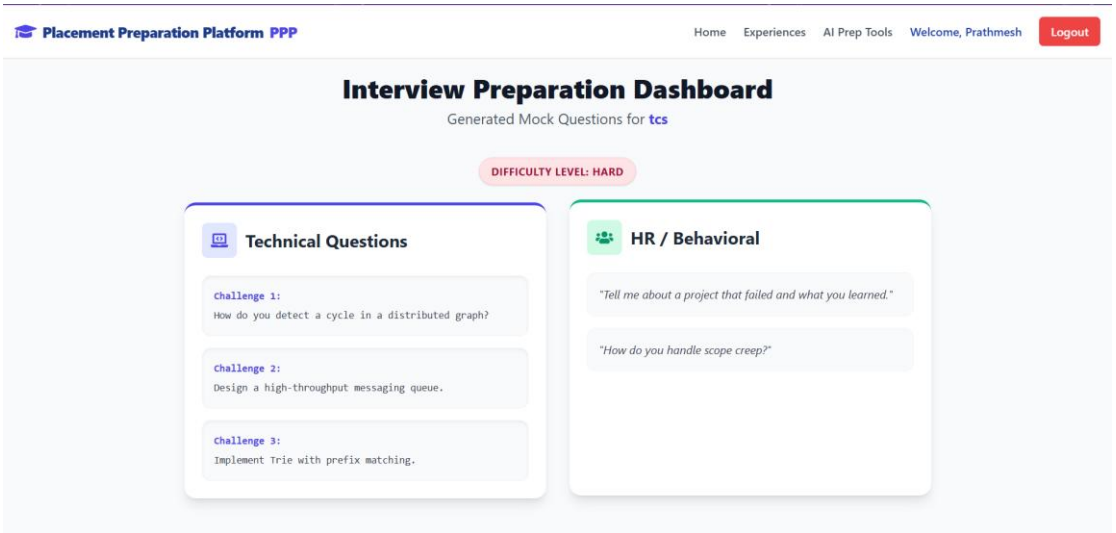


Figure 4. Interview Preparation Dashboard

4.Experiences Page

This page allows users to learn from others or share their own interview experiences. On the left side, a "Share Experience" form is provided where users can enter the company name along with aptitude, coding, and HR questions. On the right side, the

Recent Submissions section displays experiences shared by other users (e.g., Amazon), including different types of questions. The layout is divided and organized, making it easy to both contribute and explore interview experiences.

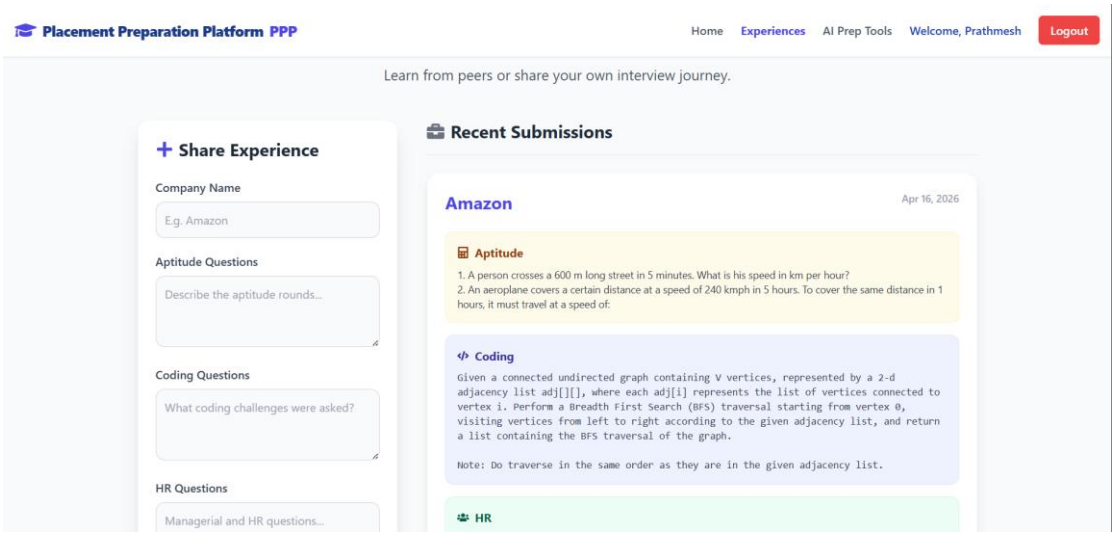


Figure 5. Experiences Page

5.AI Prep Tools Page

This page provides AI-based tools to improve interview preparation. Users can switch between features like Answer Evaluator and Resume Analyzer.

The Answer Evaluator section allows users to enter an interview question and their response. After submitting, the AI Evaluation panel displays feedback on the answer. The layout is simple and helps users practice and refine their responses effectively.

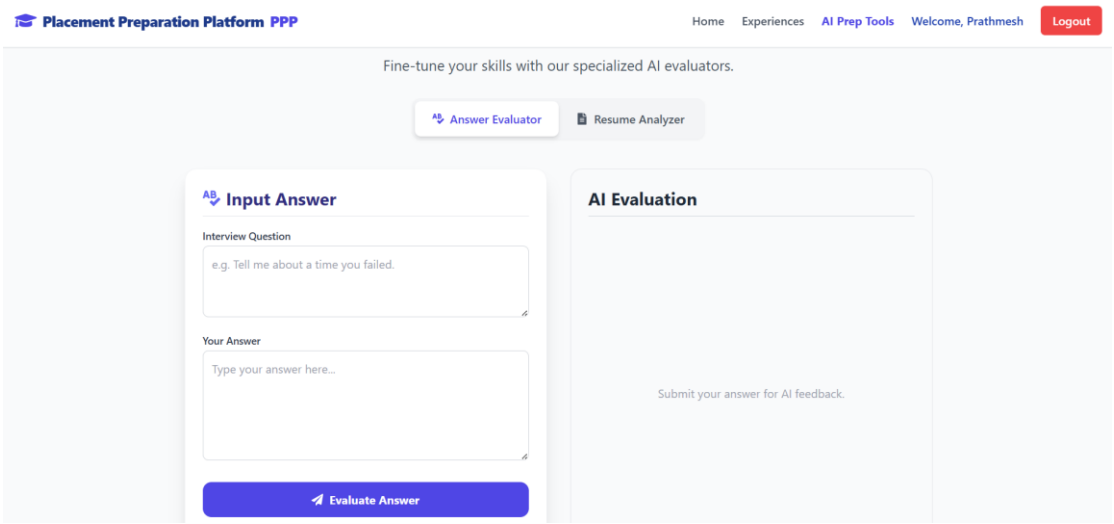


Figure 6. AI Prep Tools Page

c. Test Cases

Test Case ID	Description	Input	Expected Output	Status
TC01	User Registration	Valid name, email, password	Account created, redirect to Login	Pass
TC02	User Login	Correct email & password	JWT token issued, redirect to Dashboard	Pass
TC03	Invalid Login	Wrong password	Error: Invalid credentials	Pass
TC04	Protected Route	Access /dashboard without token	Redirect to /login	Pass
TC05	Fetch Questions	subject=DSA, difficulty=Easy	10 DSA questions returned	Pass
TC06	Quiz Submission	All 10 MCQs answered	Score calculated, result stored	Pass
TC07	Mock Test Timer	Test duration = 30 min	Auto-submit on timer expiry	Pass
TC08	Empty Quiz Answer	Submit without answering	Warning shown, no submission	Pass

Table 4 – Test Cases

CHAPTER 5: CONCLUSION

One goal stands clear. A place where engineering students prepare for placements, all in one spot. Not scattered files or distant resources, but something organized, active, learning that moves with them. Study notes live here. Practice questions appear through quick quizzes. Mock exams run on a clock, just like real ones. Built using tools common in tech today. The front glows with React.js. Server logic hums via Node.js and Express.js. Data tucks neatly inside MongoDB. Design breathes easily thanks to TailwindCSS. Everything connects without noise. No extra layers. Just function shaped by need. One thing led to another when the team worked through every phase of building software, starting with figuring out what users needed and shaping how things would look. Moving piece by piece, they built interfaces that respond, tied them to backend routes structured around standard patterns. Instead of skipping steps, each part connected - data flowed using lightweight tools made for speed. A fast bundler took charge early on, making sure changes showed up instantly during work sessions. Page transitions happened smoothly because routing ran quietly behind the scenes. Communication with servers relied on a consistent client, handling calls without extra weight. Rules for data shape were enforced before anything moved forward, catching mismatches ahead of time. Security checks slipped into place just where they belonged, never loud but always present. Tests followed along like careful notes at the edge of each feature. Tools chosen matched current habits seen across active development teams today.

It turns out the platform actually does something missing among free prep tools - mixing lessns, exercises, and performance tracking all in one place you can easily reach. When tested, each main part of the app worked as intended while unusual inputs were managed without issues.

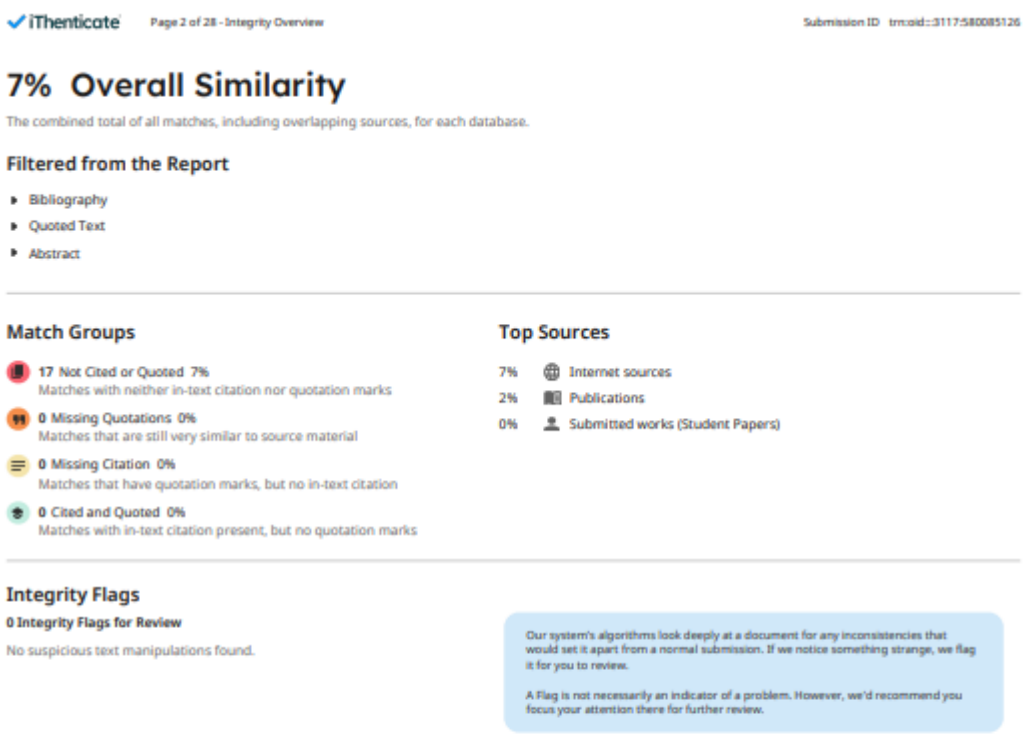
Ultimately, this placement prep tool works well now, scales when needed, yet matters most because it helps real students. Built by developers who know front-end and back-end systems, its foundation stands solid. Down the road, features like live coding practice might appear, guided tips shaped by smart algorithms could join, tailored paths for target companies may form too. Growth feels likely. The whole thing? Could become something wide-reaching, useful, deeply practical for learners everywhere.

REFERENCES

a. React. (2024). *React – The Library for Web and Native User Interfaces*. Retrieved from https://react.dev
b. Node.js Foundation. (2024). *Node.js Official Documentation*. Retrieved from https://nodejs.org/en/docs
c. MongoDB Inc.. (2024). *MongoDB Documentation*. Retrieved from https://www.mongodb.com/docs/
d. Tailwind CSS. (2024). *Tailwind CSS Official Documentation*. Retrieved from https://tailwindcss.com/docs
e. Vite. (2024). *Vite Official Guide*. Retrieved from https://vitejs.dev/guide/
f. Express.js. (2024). *Express — Node.js Web Application Framework*. Retrieved from https://expressjs.com/
g. React Router. (2024). *React Router Documentation*. Retrieved from https://reactrouter.com/
h. Mongoose. (2024). *Mongoose ODM Documentation*. Retrieved from https://mongoosejs.com/docs/

Chapter 6 -Annexure:

a. Plagiarism Report



iThenticate Page 2 of 28 - Integrity Overview Submission ID trn:oid:::3117:580085126

7% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Abstract

Match Groups

17 Not Cited or Quoted 7%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

7% Internet sources
2% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

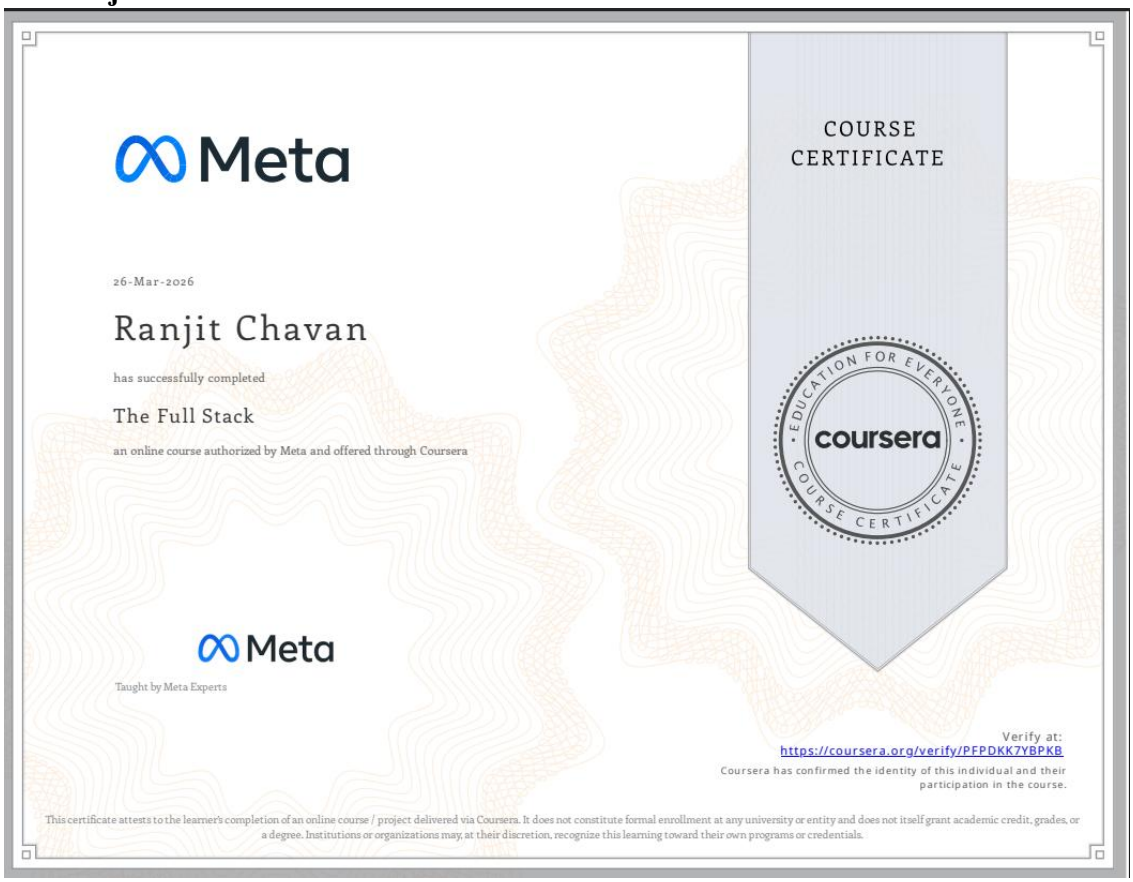
No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

b. Online Certificate:

Ranjit Chavan



Prathamesh Doiphode

Sustainability Development Goals:

Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering (PCCoE)
(An Autonomous Institute)
Affiliated to Savitribai Phule Pune University(SPPU)
ISO 21001:2018 Certified by TUV SUD

Sustainability Development Goals Mapping

Department: Computer Engineering **A.Y.: 2025-26**
Subject: Full Stack Development Lab **Sem: 2**
Mini Project Title: Placement Preparation Platform

Sr.no	Name	PRN
1.	Ranjit Chavan	123B1B103
2.	Prathmesh Doiphode	123B1B110

Class and Div: T.Y Comp B

Sr. No.	SDG No.	SDG	Description
1	SDG 4	Quality Education	The platform provides AI-powered learning resources, aptitude practice, and interview preparation, improving access to quality education for students.
2	SDG 8	Decent Work and Economic Growth	Helps students prepare effectively for placements, increasing employability and contributing to better job opportunities.
3	SDG 9	Industry, Innovation and Infrastructure	Uses AI technology to create an innovative digital platform for career preparation and industry insights.
4	SDG 10	Reduced Inequalities	Provides equal access to placement preparation resources regardless of students' background or college tier.

Ranjit M. Chavan
Prathamesh B. Doiphode
Student's Name and Signature

S. B. Galande
Mr.Shailesh B. Galande
Project guide Name & Signature